CarrySelect-4: A Registered 4-bit Carry-Select Adder from RTL to GDSII in SKY130 with a Logical-Effort Study of a Full-Custom Mirror Full Adder

Nguy Quoc Trung (23CE.B026) and Nguyen Minh Anh Tu (23CE085)
Group 7, CE2032 VLSI Design, Faculty of Computer Engineering and Electronics
Vietnam–Korea University of Information and Communication Technology (VKU), Da Nang, Vietnam
trungnq.23ceb@vku.udn.vn, tunma.23ce@vku.udn.vn

Abstract—We built CarrySelect-4, a 4-bit carry-select adder (CSLA) behind a valid/ready interface, and took it from RTL to GDSII on SkyWater SKY130 with open-source tools only. The adder has a 2-bit ripple block and two speculative 2-bit blocks, and the low carry selects between their results. A three-state Moore controller registers the inputs and outputs, which gives a latency of 2 cycles and one result every 2 cycles. Self-checking simulation passes 512 of 512 input vectors. Synthesis with Yosys gives 218 NAND2 equivalents. After place and route with LibreLane at 100 MHz, setup and hold slack are positive in all nine signoff corners and DRC and LVS report no errors. In a separate branch we sized, laid out and extracted a 28-transistor mirror full adder in Magic. A Logical Effort estimate of its ripple-carry path, 312 ps, falls between the schematic SPICE result (264 ps) and the extracted one (420 ps).

Index Terms—carry-select adder, SKY130, OpenLane, LibreLane, logical effort, full-custom layout, mirror adder

I. Introduction

In a ripple-carry adder (RCA) the carry passes through every bit, so the delay grows as $O(n)$ with the word length n. A carry-select adder [1] shortens this path. It computes the upper bits twice, once for each possible incoming carry, and a multiplexer picks the right result once the real carry is known. The price is extra area [2]. We wanted to see what this trade looks like in a real flow, so we built a 4-bit instance and followed it through RTL and verification, through synthesis and static timing with Yosys [3] and OpenSTA [4], and through place and route with OpenLane 2 / LibreLane [5], [6] on SKY130 [7]. The second half of the project goes down to transistor level. We drew the adder's leaf cell, a mirror full adder, in Magic [8] and checked how close a hand estimate with Logical Effort [9] comes to SPICE before and after layout.

II. Architecture

Fig. 1 shows the top level. A 9-bit input word $\{c_{in}, b, a\}$ is captured in an input bank of 9 flip-flops when ld_en = valid_in & ready_out. The CSLA splits its six full adders into three 2-bit RCAs. The low block adds $a[1:0] + b[1:0] + c_{in}$. The hi0 and hi1 blocks both add $a[3:2] + b[3:2]$, one with a carry-in of 0 and one with

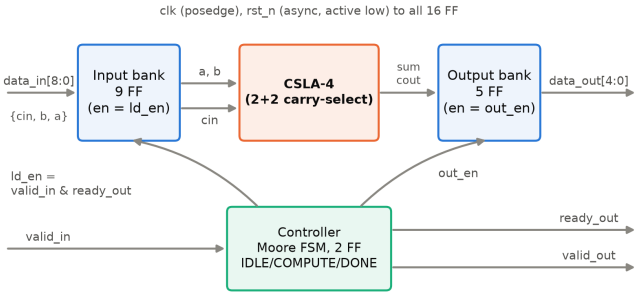


Fig. 1. Registered CarrySelect-4 datapath and controller.

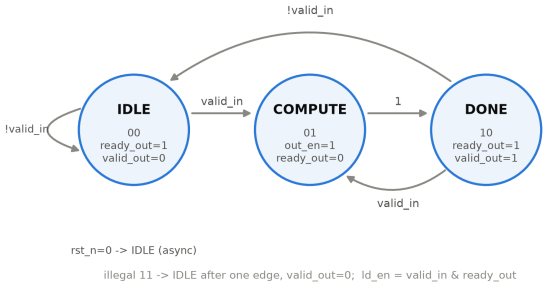


Fig. 2. Moore controller: state codes, outputs and transitions.

1. The carry c_2 out of the low block drives the select inputs of three 2:1 multiplexers, which produce sum[3:2] and cout. The full adder and multiplexer modules keep their hierarchy through synthesis, because otherwise ABC is free to rebuild the netlist as a different adder. The 5-bit result goes into an output bank of 5 flip-flops. A Moore controller with two state bits (Fig. 2) steps through IDLE, COMPUTE and DONE, and the unused state code falls back to IDLE. All 16 flip-flops use an asynchronous active-low reset.

TABLE I
Verification cases (Verilator, self-checking)

Case	Scenario	Result
1	directed vectors after reset	PASS
2	back-to-back, latency and throughput	PASS
3	random valid_in bubbles (30%)	PASS
4	async reset in COMPUTE and in DONE	PASS
5	exhaustive, 512 vectors, 0 errors	PASS
–	illegal state 2'b11	PASS

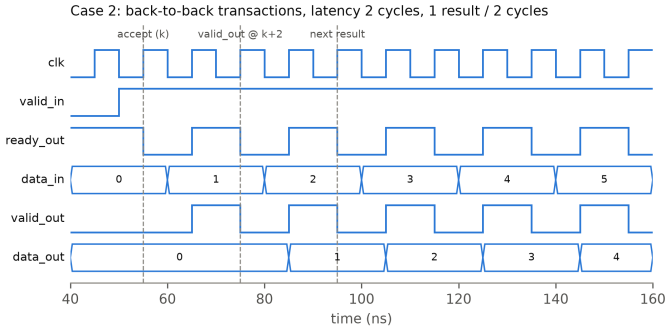


Fig. 3. Case 2 waveform: latency of two cycles, one result every two cycles.

III. Verification

The testbench is self-checking SystemVerilog running on Verilator. A scoreboard compares each result with vectors from an independent Python golden model and also checks that the latency is exactly two cycles. Table I lists the cases. In Fig. 3 an input accepted at clock edge k gives valid_out at edge $k+2$, and the next result arrives two cycles after that.

IV. Synthesis and Physical Design

Yosys maps the design onto sky130_fd_sc_hd cells. The result is 49 cells and 817 μm^2, or 218 NAND2 equivalents against a budget of 500. We constrained a 10 ns clock with 2 ns input and output delays. OpenSTA then reports a worst setup slack of 6.78 ns at the typical corner. The register-to-register critical path, a_q[0] -> cout_q, arrives after 1.78 ns.

Fig. 4 compares the CSLA with a 4-bit RCA. With the hierarchy kept, the CSLA is faster (1.24 vs. 1.71 ns) and larger (236 vs. 135 μm^2), so its area–delay product is worse. When ABC may flatten the logic, both adders shrink and slow down. The flat CSLA is still the faster one (1.59 vs. 2.00 ns) and has the best area–delay product. We did not isolate why the speed-up is small at this width; the duplicated upper block is the likely cost. The flat ranking also depends on the tool version: an earlier run with Yosys 0.70 in another environment ordered the two flat netlists the other way round, so we base our conclusion on the hierarchical comparison.

LibreLane places and routes the design with a 55% core-utilization target and an aspect ratio of 1 (Fig. 5); Table II

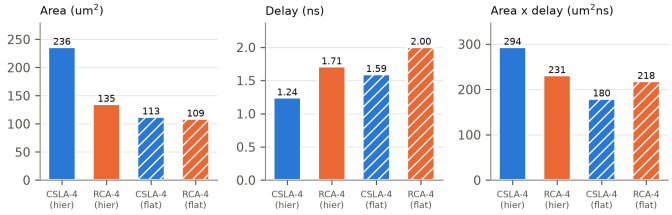


Fig. 4. CSLA-4 vs. RCA-4 after synthesis (typical corner); hatched bars are flattened netlists.

TABLE II
Post-route signoff at 100 MHz

Metric	Value
Die area (µm^2)	3340
Std cells / final utilization (%)	123 / 80.7
Setup slack, tt / worst corner (ns)	6.42 / 5.21
Hold slack, tt / worst corner (ns)	0.384 / 0.151
f_{max} estimate, tt (MHz)	280
DRC Magic / KLayout, LVS errors	0 / 0 / 0
Antenna violations	0
Total power (mW), wirelength (µm)	0.363, 1587

gives the signoff numbers. Setup and hold slack are positive in all nine corners (min/nom/max extraction × ss/tt/ff), including the recovery and removal checks on rst_n. At the typical corner the estimated maximum frequency is 280 MHz. A second run at a 5 ns clock meets setup and hold in all 9 corners; the worst setup slack is 0.211 ns.

V. Full-Custom Full Adder and Logical Effort

The leaf cell is a 28-transistor mirror adder. A majority (carry) gate produces $\overline{C_o}$, the sum gate reuses that signal to produce $\overline{S}$, and two inverters restore the polarity. We simulated an FO4 inverter in ngspice at the typical corner and obtained $\tau = 12.9$ ps (FO4 = 64.5 ps) and $p_{inv} = 1.13$ for $W_p/W_n = 0.840/0.420$.

We analysed the 2-bit ripple path inside an RCA block. It runs from a_0 through two carry gates and two inverters to c_2, which drives three multiplexer select pins ($C_{load} = 3\,C_{S,\mathrm{mux2}}$). We took $g_{carry} = 2$ and $g_{inv} = 1$, and read the branching effort from the transistor widths. Then

$$F = GBH, \qquad \hat{f} = F^{1/N}, \qquad D = N\hat{f} + P, \tag{1}$$

with $G = 4$, $B = 31.5$, $H = 1.45$ and $N = 4$, so that $F = 183$ and $\hat{f} = 3.68$. The parasitic term is $P = 2p_{carry} + 2p_{inv}$, and we simulated $p_{carry} = 3.60$. This gives $D = 24.2\,\tau = 312$ ps. We tried two sizings. S1 scales each transistor by its stack depth. S2 also gives the carry-out inverter the size that Logical Effort assigns to the last stage. S2 was faster in SPICE (264 vs. 307 ps), so we laid it out.

In the layout (Fig. 6) the shared polysilicon columns follow Euler paths, which leaves a single diffusion break in the pMOS and nMOS rows. Full-width n- and p-tap rows under the supply rails keep every device within a few micrometres of a tap, well inside the 15 μm latch-up

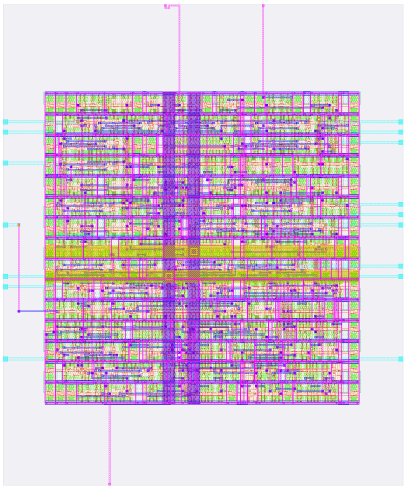


Fig. 5. Routed layout of carryselect4_top (LibreLane, SKY130).

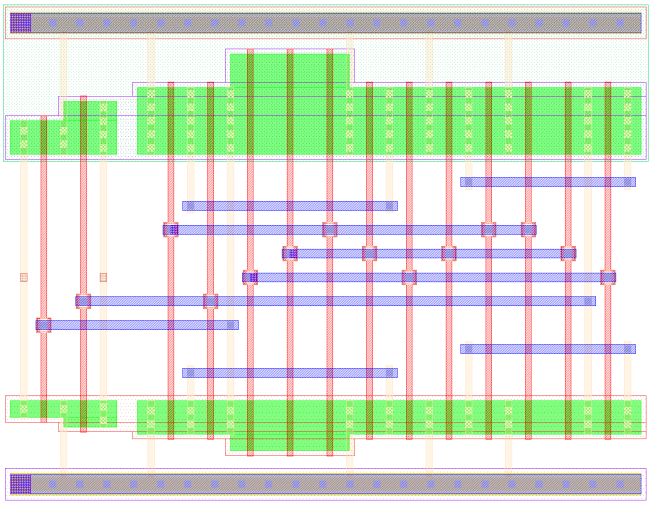


Fig. 6. Full-custom 28T mirror full adder in Magic (DRC and LVS clean).

TABLE III
Ripple-carry path delay $a_0 \rightarrow c_2$

Method	Delay (ps)	LE error
Logical Effort	312	–
SPICE, schematic	264	18.2%
SPICE, extracted	420	-25.7%

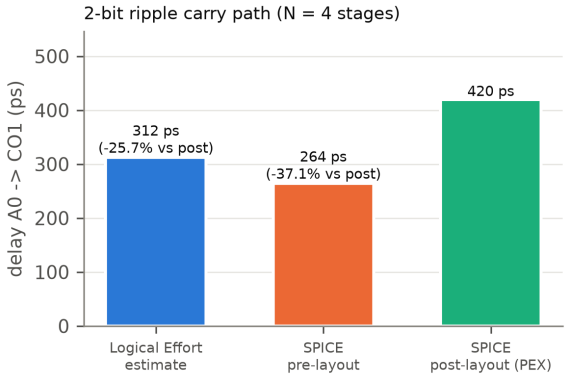


Fig. 7. Logical Effort estimate vs. SPICE before and after extraction.

limit. The cell measures $16.2 \times 12.3\,\mu$m ($200\,\mu$m^2). That is about ten times the $20.02\,\mu$m^2 of the library's fa_1 cell. It is DRC-clean with the full SKY130 deck and matches the schematic in Netgen LVS. Table III and Fig. 7 compare the estimate with SPICE. Against the schematic simulation the estimate is off by 18.2% (too slow). Against the extracted netlist it is off by -25.7% (too fast). The schematic netlist has no source/drain junction capacitance, while extraction adds it together with 80 wiring capacitors. Logical Effort folds all of this into one parasitic term per stage.

VI. Conclusion

CarrySelect-4 reached its functional and timing targets. Exhaustive simulation passes, the routed design closes at 100 MHz in every corner with clean DRC, LVS and antenna checks, and it uses less than half of its area budget. At four bits the carry-select structure is faster than ripple carry but loses on area–delay product unless synthesis flattens it. In the full-custom study, Logical Effort picked the faster of the two sizings and gave a delay inside the pre- and post-layout band. It cannot give an accurate absolute delay without a layout-aware parasitic term. The custom cell is a study cell and is not used in the standard-cell flow.

Three limitations apply. The custom adder is much larger than the library cell: we did not optimise its area, and its routing uses nine horizontal metal-1 tracks. The pre-layout simulation had no diffusion capacitance (no ad, as, pd, ps on the devices), so the gap between the pre- and post-layout results is likely larger than the layout alone would cause. And the ranking of the flattened adders depends on the synthesis tool version. As next steps we would place the custom adder in the flow as a macro with LEF and LIB views generated from its layout, and compare wider adders (8 to 32 bits, with unequal block sizes), where carry-select should pay off more clearly.

References

[1] O. J. Bedrij, "Carry-select adder," IRE Transactions on Electronic Computers, vol. EC-11, no. 3, pp. 340–346, 1962.

[2] N. H. E. Weste and D. M. Harris, CMOS VLSI Design: A Circuits and Systems Perspective, 4th ed. Addison-Wesley, 2010.

[3] C. Wolf, "Yosys open SYnthesis suite," https://yosyshq.net/yosys/, 2013.

[4] Parallax Software, "OpenSTA: Static timing analyzer," https://github.com/The-OpenROAD-Project/OpenSTA, 2019.

[5] M. Shalan and T. Edwards, "Building OpenLANE: A 130nm OpenROAD-based tapeout-proven flow," in Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2020.

[6] LibreLane Contributors, “LibreLane: an RTL-to-GDSII flow (successor of OpenLane 2),” https://github.com/librelane/librelane, 2025.
[7] SkyWater Technology and Google, “SkyWater SKY130 open source process design kit,” https://github.com/google/skywater-pdk, 2020.
[8] J. K. Ousterhout, G. T. Hamachi, R. N. Mayo, W. S. Scott, and G. S. Taylor, “Magic: A VLSI layout system,” in Proc. 21st Design Automation Conference (DAC), 1984, pp. 152–159.
[9] I. Sutherland, B. Sproull, and D. Harris, Logical Effort: Designing Fast CMOS Circuits. Morgan Kaufmann, 1999.